SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

Many small and medium enterprise (SME) owners face difficulties in making informed financial decisions due to fragmented data and limited analytical capabilities. Financial information is often spread across different sources such as bank statements, invoices, and business documents, making it hard to get a clear picture of the business's overall financial health. As a result, important decisions are frequently made based on assumptions rather than accurate insights, which can lead to cash flow issues, overspending, or missed opportunities.

To address these challenges, this project proposes an AI-powered decision intelligence system that supports SME owners in managing their finances more effectively. The system integrates both structured data, such as financial records and expenses, and unstructured data, such as invoices and documents, to provide a more comprehensive understanding of the business. By analysing this information, the system is able to generate meaningful insights that reflect the current financial condition of the business.

In addition, the system goes beyond basic data analysis by providing decision support features that help users take appropriate actions. These include predicting cash flow runway, explaining changes in spending patterns, identifying useful information from uploaded documents, and simulating potential business decisions through a "What-If" feature. Each recommendation is supported with clear explanations and measurable outcomes, allowing users to understand not only what decision to make, but also why it is suggested.

Overall, the system aims to empower SME owners to make more confident, data-driven decisions, reduce financial risks, and improve the sustainability of their business operations.

1.1. Purpose:

The purpose of this document is to describe the technical scope and design decisions behind the development of the AI-powered SME decision intelligence system. It explains how the system is structured, how data is processed, and how the main features work together to support SME owners in making better financial decisions. This document ensures that all parts of the system are clearly defined and aligned with the overall project goals.

So, the key element of system that has been covered are as followed:

1. **Architecture**
   - The system uses a layered architecture consisting of frontend, backend, and AI services. The frontend handles user input and displays results, while the backend manages processing and communication between components.
   - The system includes a simulation engine that calculates financial outcomes based on user inputs such as cash balance, expenses, and invoices. It processes data before sending it for further analysis.
   - The system integrates an AI explanation module using a language model (GLM). This module converts simulation results into clear, human-readable explanations and recommendations.
   - The system uses API-based communication between frontend, backend, and AI services. All data exchange and processing requests are handled through APIs.
   - The system is built in a modular structure where each feature (simulation, prediction, explanation, and recommendation) works as a separate component within the same system.

2. Data Flows

The data flow starts when the user inputs financial information such as cash balance, expenses, and invoices through the system's frontend. This input is then sent to the backend through an API request. The backend processes the data and passes it to the simulation engine, where financial calculations are performed based on the selected what-if scenario. After that, the simulation results are forwarded to the AI explanation module to generate insights, recommendations, and interpretations. Finally, the processed output is sent back to the backend and returned to the frontend, where it is displayed in the dashboard for the user.

3. Model Process

The model process begins when the user selects a financial scenario such as delaying payment, collecting invoices, or taking a loan. The system then takes the user input and runs it through the simulation engine to calculate the financial outcome. After that, the results are passed to the AI module to generate explanations and recommendations based on the scenario. The system then combines the simulation output and AI insights to produce a final decision. This final output is shown to the user in the dashboard, including the result, explanation, and impact.

4. Role of Reference

This document is used as a reference for developers and testers during the system development process. It provides a clear description of the system structure, data flow, and model process. It is also used to ensure that all components of the system are developed and tested according to the same understanding and requirements.

1.2. Background

Many SME owners face difficulties in managing cash flow and making financial decisions due to limited visibility of their future financial situation. Decisions are often made based on manual calculations or assumptions, which can lead to cash shortages or poor financial planning. The system is designed to help SME owners simulate financial decisions before they are executed.

1. **Previous Version**

   In the previous approach, SME financial management is typically done using spreadsheets or manual tracking methods. Users calculate cash flow, expenses, and invoices separately without any integrated system. There is no simulation or predictive tool available to show the impact of financial decisions before they are made.

2. **Changes in Major Architectural Components**

   The new system introduces a simulation engine that calculates financial outcomes based on user inputs such as cash balance, expenses, and invoices. It also introduces an AI explanation module that converts simulation results into clear financial insights and recommendations. A decision-flow structure is added to connect all components in a structured process from input to final output. These components are integrated using API-based communication between frontend, backend, and AI services.

3. **New capabilities that are only introduced in this**

   The system introduces a chatbot that allows users to test financial decisions such as delaying payments, collecting invoices, or taking loans before applying them in real life. It provides simulated results including cash flow changes, risk level, and recommended actions. The system also generates explanations for each outcome and shows the financial impact clearly to support better decision-making.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
SME Owner	Uses the system to input financial data such as cash balance, expenses, invoices, and tests what-if scenarios before making decisions	Clear financial insights, easy-to-understand recommendations, and reliable simulation results to support decision-making
Financial Manager	Monitors financial data, reviews system outputs, and supports decision-making for the business	Accurate analysis, fast simulation results, and clear explanation of financial impact
Business Consultant	Uses system insights to advise SME owners on financial strategies and decisions	Reliable recommendations, understandable explanations, and scenario comparison features
Development Team	Develops, maintains, and improves the system including AI, backend, and simulation engine	Clear requirements, stable system architecture, and well-defined API structure
QA Team	Tests system functionality including simulation accuracy, AI output, and system flow	Clear test cases, correct output validation, and full edge case coverage

2. System Architecture & Design

2.1. High Level Architecture

Overview

Type	Details
System	Web Application (Responsive Dashboard for SME users)
Architecture	Cloud-based layered architecture with AI service integration

The system is structured using a client-server architecture where the SME owner interacts with a web-based dashboard. The frontend sends user inputs such as cash flow, expenses, and invoices to the backend through API calls. The backend processes the data and connects to two main components: the simulation engine and the AI service layer. The simulation engine calculates financial outcomes, while the AI layer generates explanations and recommendations. All processed results are returned to the frontend as a decision dashboard showing scenario results, risk level, and impact.

LLM as Service Layer

The system integrates a multi-layer AI service pipeline consisting of Google Gemini as the primary LLM, Z.AI GLM as a secondary fallback model, and a local rule-based system for failure handling.

The process begins when the backend constructs a structured prompt using user inputs such as cash balance, expenses, invoices, and selected what-if scenarios. This prompt also includes simulation results such as cash difference, risk level, and timeline impact. The prompt is then sent to the AI service layer as part of the context window.

The AI service layer processes the input and returns a structured response containing explanation, recommendation, and reasoning. The system then parses this response in the backend and maps it into the output format for the dashboard.

Token limits are handled by summarising financial inputs before sending them to the AI service layer, ensuring only relevant and compact data is included in the prompt. The AI response is then validated and formatted before being passed to the frontend.

Dependency Diagram

- The system constructs prompts at the backend by combining user-provided financial inputs and the results generated from the simulation engine. These inputs include cash balance, expenses, invoices, and selected what-if scenarios. The backend formats this information into a structured prompt that clearly represents the financial situation and decision context. This structured prompt is then sent to the AI service API via an API request for further processing.

- The context window includes only relevant and summarized data to ensure efficiency and accuracy in AI processing. The following elements are included:
  - Financial summary (cash balance, expenses, invoices)
  - Selected what-if scenario
  - Simulation results (predicted cash flow, risk level)

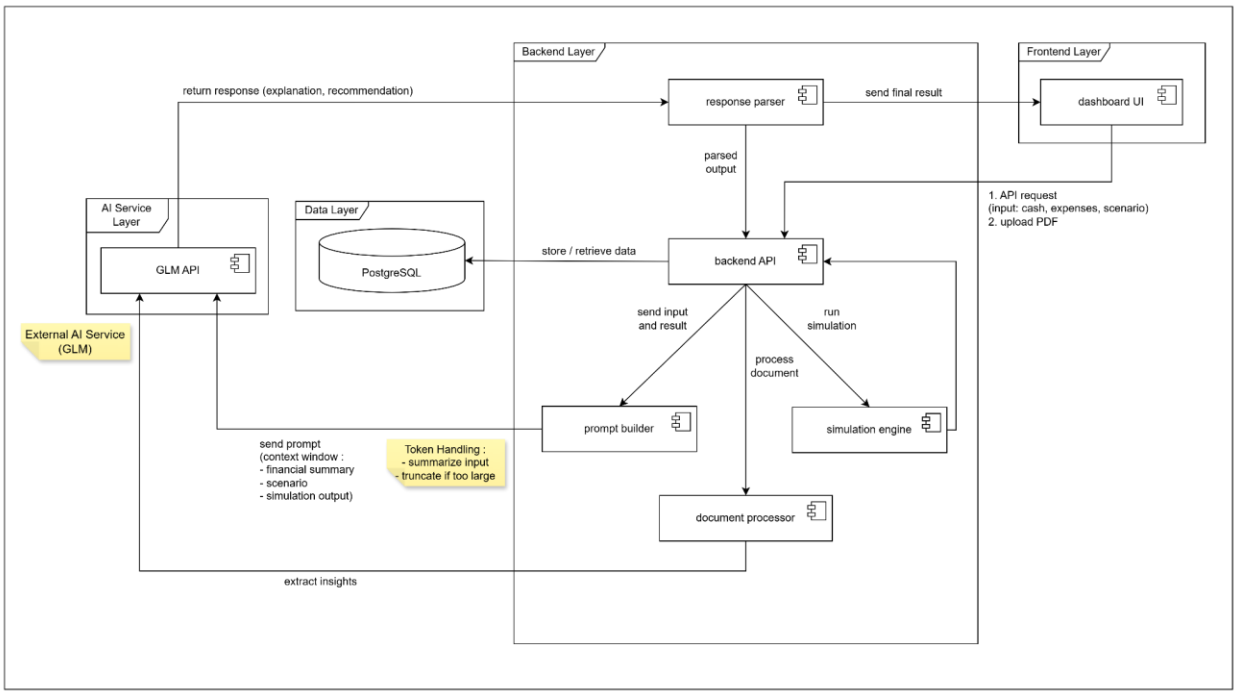
- After processing the prompt, the AI service API returns a response containing explanation, recommendation, and risk insights. The backend system receives this response and parses it into structured components. These components are then formatted appropriately and passed to the frontend dashboard for user display. This ensures that the output is clear, readable, and useful for decision-making.

- Token limitation is managed at the backend level before sending data to the AI service API. Large inputs such as CSV files or lengthy documents are first processed and summarized. Only key financial information is extracted to reduce size. If the input still exceeds the token limit, truncation or chunking is applied to ensure that the request remains within acceptable limits for the AI model.

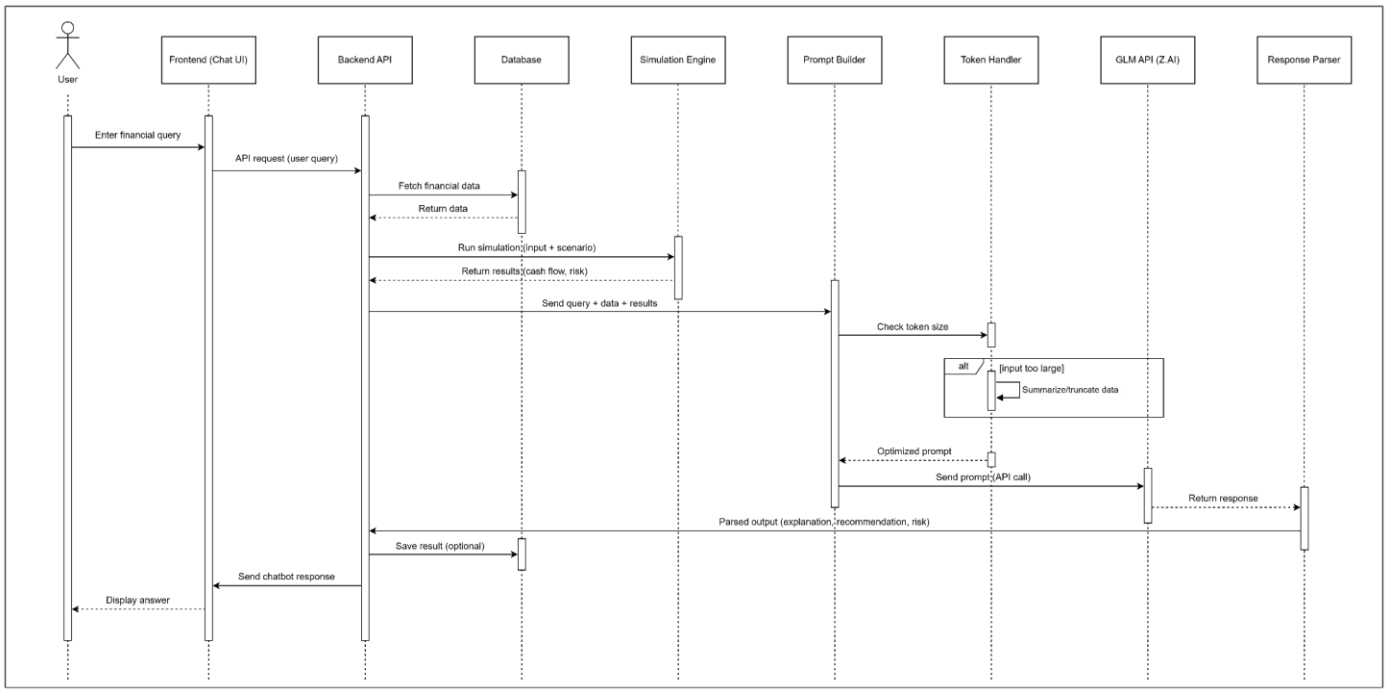
- The system relies on several key API interactions:
  - The frontend dashboard sends user inputs, including financial data, selected scenarios, and uploaded files, to the backend API through an API request.
  - The backend API sends a request to the simulation engine to perform financial calculations based on the provided inputs and selected scenario.
  - The simulation engine returns the computed results, such as predicted cash flow and risk indicators, back to the backend API.
  - The backend API constructs a prompt and sends it to the AI service API for generating AI-based explanations and recommendations.
  - The AI service API processes the prompt and returns a response containing explanation, recommendation, and risk insights to the backend API.
  - The backend API communicates with the database (PostgreSQL) to store and retrieve financial data, simulation results, and user-related information.
  - The backend API sends uploaded documents, such as PDF or CSV files, to the document processor for preprocessing and data extraction.
  - The document processor forwards the extracted or summarized content to the AI service API for further analysis and insight generation.
  - The AI service API returns the processed document insights to the backend API, which are then prepared for display to the user.

- The frontend dashboard acts as the user interface where financial data is entered and results are displayed. It communicates with the backend API, which serves as the central controller of the system. The backend coordinates with internal services such as the simulation engine for financial calculations and the document processor for handling uploaded files. It also communicates with the external AI service API to generate AI-based explanations and recommendations. All relevant data is stored and retrieved from the database, ensuring persistence and consistency across the system.

- The system is designed as a distributed architecture where different components operate independently but are connected through API communication. The main components include:
  - Frontend (Dashboard UI)
  - Backend API (application server)
  - Simulation Engine (financial logic processing)
  - AI Service (AI service external API)
  - Document Processor (file handling service)
  - Database (PostgreSQL for data storage)



Sequence Diagram

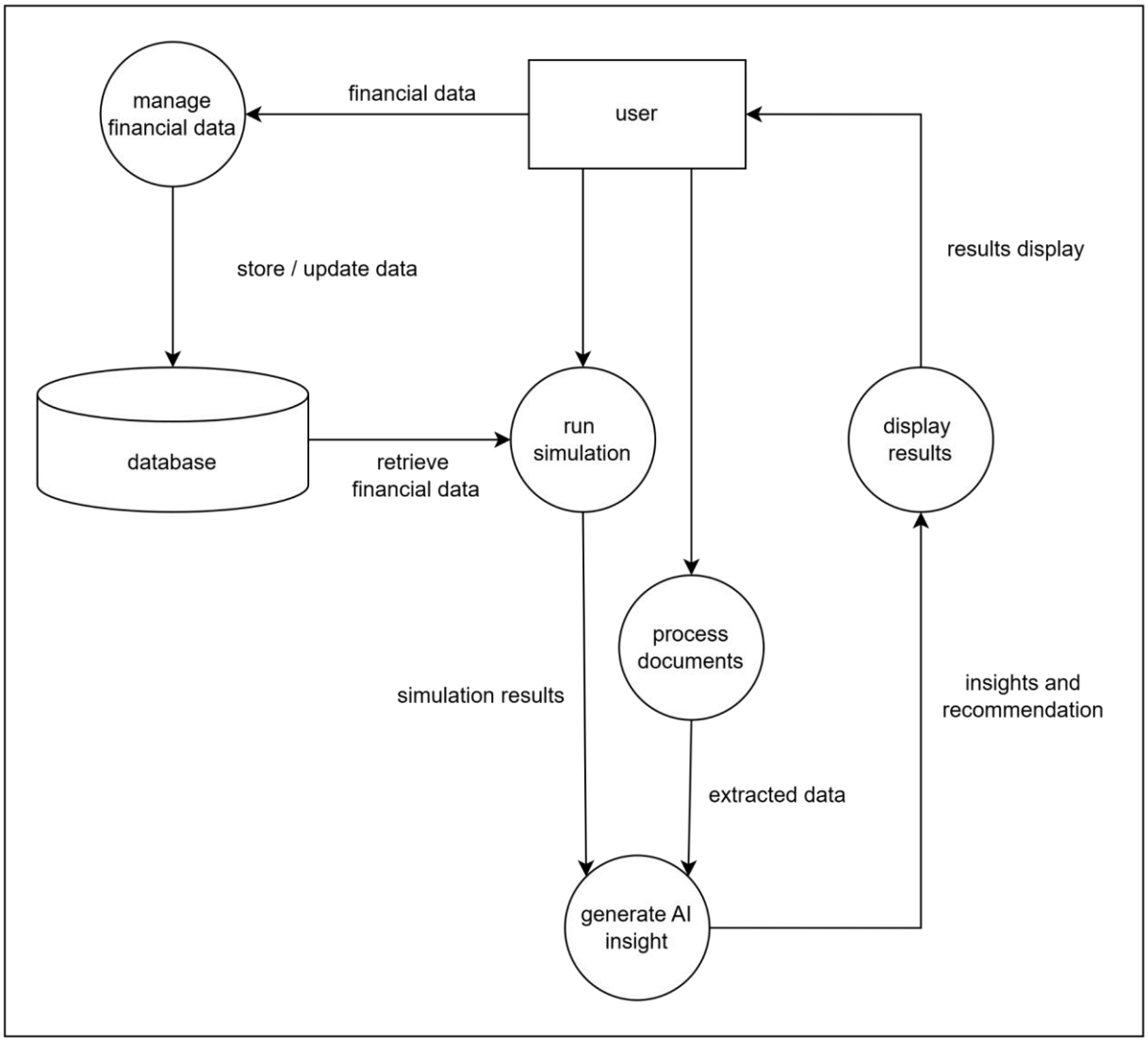


2.2. Technological Stack

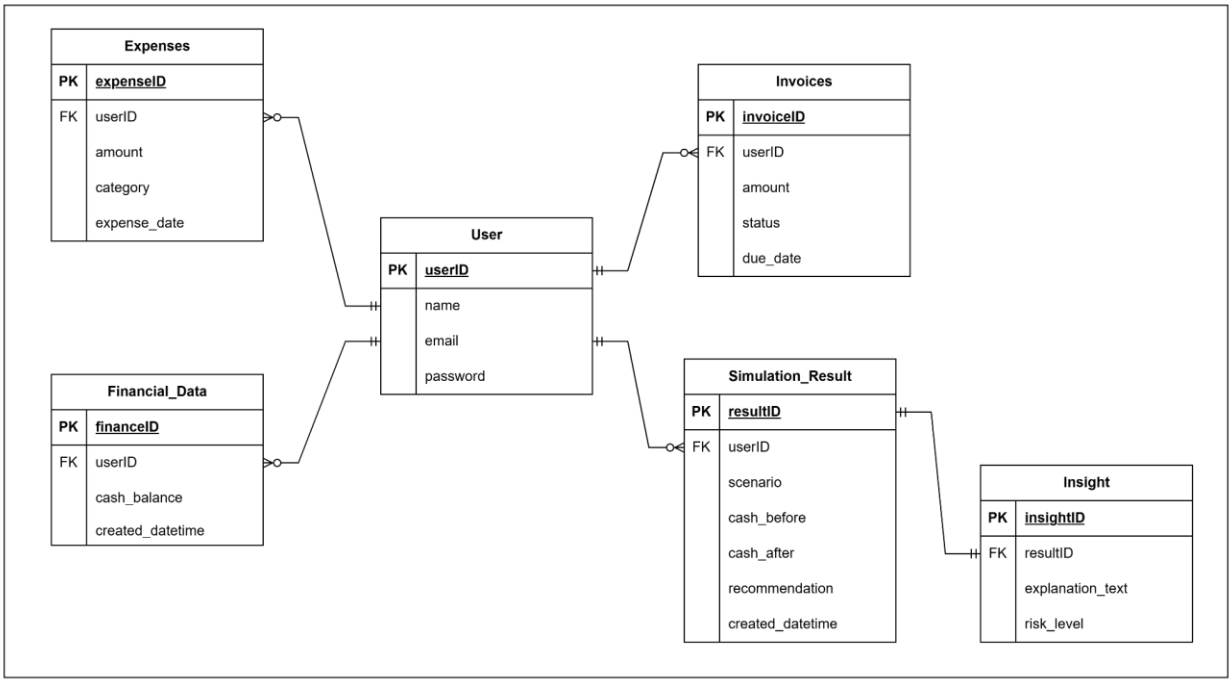
- **Frontend**
  Streamlit, interactive AI dashboard with dynamic UI and state management
- **Backend**
  Python 3.14, powered by Gemini 2.5 Flash API for document analysis and explainable AI
- **Database**
  PostgreSQL, semantic search and structured data storage
- **Cloud/Deployment**
  Streamlit Community Cloud + GitHub, deployment with secure API key management

2.3. Key Data Flows

- **Data Flow Diagram (DFD)**



- **Normalized Database Schema**



3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

	Features	**Description**
1	Cash Flow Runway Predictor	Calculates business survival duration based on cash balance, revenue, expenses and seasonal adjustment factor. Outputs runway estimate and financial health status.
2	AI Financial Insight Generator	Processes news headlines to generate financial recommendations, risk level and RM impact estimation.
3	PDF Invoice Scanner	Extracts structured invoice data (vendor,amount, tax, due date, PO reference) from digital PDFs.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Reliability	System uptime must be ≥ 99% and system must return a fallback response within ≤ 2 seconds if AI API fails or returns invalid output.	Implements fallback system using rule-based extraction and local computation logic.
Performance	Simulation response time must be ≤ 800ms and AI response time must be ≤ 1500ms under normal load (≤ 50 users).	Uses lightweight model that reduced prompt size and token optimization.
Cost Efficiency	Each AI request must use ≤ 1500 tokens and total API usage must stay within defined budget (e.g. RM200/month).	Uses structured prompts, filtered inputs and avoids sending unnecessary data.
Accuracy	Financial calculations must be 100% accurate. AI output must achieve ≥ 80% correctness based on test cases.	Strict prompt constraints with fallback rule-based logic with JSON schema enforcement.
Usability	At least 90% of users can understand system output without assistance during testing.	Uses UI explanations, risk labels and RM-based impact visualisation.

3.3. Out of Scope / Future Enhancements

The following features are not implemented in the current MVP but are planned for future development:

- Live bank API integration (Maybank, CIMB, etc.) for real-time financial data
- User authentication and login system
- Database storage for historical invoices and financial decisions
- Automated payment execution or transaction processing
- OCR support for scanned or image-based receipts
- Multi-currency financial analysis beyond Malaysian Ringgit (RM)
- Multi-user collaboration system with shared financial dashboards
- Long-term financial forecasting using historical dataset accumulation

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

- **Grayscale Rollout & A/B Testing**

The system adopts a grayscale rollout strategy where all the features, including the cash flow runway predictor, AI insight module, and document scanner, are first tested internally. This ensures system stability, especially for data processing accuracy and AI integration outputs.

After internal testing, the systems were released to a small group of selected users for further evaluation while monitoring the response time, accuracy and system stability. Stable components such as the cash flow runway predictor serve as a baseline for system reliability, while the AI insight module, which is limited to prompt output (≤40 words), and the document scanner, which extracts financial data in RM format, are monitored for output accuracy and consistency.

In addition, A/B testing is conducted on the AI-generated output formats by comparing a basic headline-based output with a more structured output that includes explanation and risk indication. This helps determine which format is easier for users to understand and more effective for decision-making before full deployment.

- **Emergency Rollback (ER) & Golden Release**

  In the event of system anomalies such as AI response failures, dashboard update errors, or issues in document data extraction, an Emergency Rollback (ER) mechanism is triggered. This ensures that the system can quickly revert to the last stable version to maintain usability.

  The Golden Release refers to the most stable version of the system, where core functionalities such as the cash flow runway predictor and dashboard display operate reliably without critical errors. During rollback, AI-based features such as the insight module and document scanner may be temporarily disabled, while essential financial calculations remain available to users.

  This approach ensures continuous system availability, prioritizing core financial functionality even during service disruptions.

- **Priority Matrix**

  - **P1 Critical**
    If the cash flow runway predictor fails or produces incorrect financial calculations, or if system-wide failures occur, an Emergency Rollback (ER) is triggered. The system reverts to the Golden Release to ensure core financial functionality remains available.

  - **P2 High**
    If the AI insight module or document scanner fails to generate outputs, produces errors, or becomes temporarily unavailable, a local fallback mechanism is activated. This provides simplified or non-AI-based outputs (e.g. basic simulation results or default responses) while maintaining system usability.

- **P3 Medium**

  If there are delays in dashboard updates, minor AI latency, or formatting inconsistencies, the system continues operating normally while monitoring performance. No rollback is triggered.

- **P4 Low**

  Minor UI issues or display inconsistencies that do not affect functionality are logged for future improvement and do not impact system operation.

4.2. Monitoring

The system is continuously monitored using key indicators such as response time, error rate, AI output success, and dashboard latency. These metrics help detect performance issues during system operation.

If the AI module experiences errors or becomes unavailable, a local fallback mechanism is triggered. This fallback provides predefined or simplified outputs (such as basic simulation results or cached responses) to ensure the system remains usable.

For critical failures affecting core calculations, the system may revert to the Golden Release through an Emergency Rollback (ER) to maintain system stability.

4.3. Assumptions

The system assumes that users have stable internet access when using the platform, especially during financial data input, AI interaction, and document upload processes.

- Users are expected to input accurate financial information (such as cash balance, expenses, and invoices), as the system relies on this data for cash flow calculations and simulation results.
- The AI insight module is designed to process short user inputs, limited to headlines or short prompts within 40 words. It is assumed that users provide concise and relevant input to maintain response quality and reduce token usage.
- The system assumes that all extracted financial values from the document scanner are in Malaysian Ringgit (RM), as multi-currency processing is not supported in the current version.

4.4. External Dependencies

Tools	Purpose	Risks
Google Gemini	Handles AI insight generation, recommendation output, and document interpretation	High level – Service downtime, latency, or API limits may affect AI response generation
Z.AI	Acts as a fallback AI provider when the primary model is unavailable	Medium level - Backup responses may vary in quality or response speed
API Communication Layer	Enables communication between frontend, backend, AI services, and simulation engine	Medium level – API failure may interrupt data transfer between components
PDF Document Parser	Extracts structured financial information from uploaded PDF files using the help of AI	Medium level - Unsupported or poor-quality PDFs may reduce extraction accuracy
Local Fallback Engine	Provides predefined or simplified outputs when external AI services are unavailable	Low level - Limited intelligence compared to cloud AI models

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline

Date	Activity	Justification
16/04	Opening Ceremony & Initial Task Distribution	The team attended the opening ceremony where hackathon domains and project themes were introduced. Early task distribution was also carried out to assign responsibilities across frontend, backend, and documentation.
18/04	Topic Finalization	The team finalized the project scope and feature direction.
20/04	Z.AI Workshop Participation	The team attended a workshop on Z.AI to understand the features and usage of it.
21/04	Sprint Planning	A 5-day sprint was initiated to focus on completing development and documentation tasks within the hackathon period.
23/04	API Access & Frontend Completion	API access was received from the organizers. Front-end development was completed, while the pitching video draft was prepared in parallel.
24/04	Backend Development, Documentation & Moderator Review	Backend development reached near completion while documentation work, including PRD, SAD, and QA, progressed simultaneously. The team also attended a moderator review session to receive feedback and validate project direction.
25/04	Pitching Video & Submission Preparation	Final preparations were completed, including pitching video production, system review, and organizing all required deliverables for submission.

5.2. Team Members Role

Name	Role
Dhania Nafeesa binti Mohd Dzulkaedah	**Backend Lead** – System Architecture, API Development & Simulation Logic Integration
Aina Wafiyah binti Mohd Esa	**Frontend Developer & Media Lead** – UI/UX Dashboard Implementation & Pitch Production
Aina binti Jamal	**Documentation Specialist** – Technical Writing, Requirement Analysis and System Documentations
Aqasyah Fatin Afrina binti Mokhtar	**Full-Stack Support & Technical Writer** – Backend Module Assistance & Documentation Management
Aina Alesha binti Abdul Aziz	**Backend Developer & Technical Writer** – Backend Processing & Technical Documentation

5.3. Recommendations:

- To improve scalability, the system could implement caching for repeated AI insight requests to reduce API usage and improve response time. This would help lower token consumption and improve efficiency for frequently requested outputs.
- For performance improvement, asynchronous API handling could be introduced to reduce waiting time during AI processing and document extraction. This would allow the dashboard to remain responsive while background tasks are completed.
- To improve reliability, a stronger fallback mechanism and health-check monitoring could be added to detect AI service interruptions earlier.
- To improve the system by including support for additional file formats and multi-currency extraction to increase system flexibility.